Running LOFAR imaging pipelines using Docker/Singularity [1]

The now standard LOFAR imaging pipelines (like prefactor and factor) depend on a variety of software packages and this can be hard for some users to install all the necessary software packages, and their dependencies on their machines. In this chapter, we present an overview of a docker image that comes pre-packaged with all the tools that a user would need to run the LOFAR pipelines. For other methods to install LOFAR software, see the LOFAR User Software section on the LOFAR wiki.

With this new docker framework [2], initializing your LOFAR processing framework can be as simple as

```
$ docker run --rm -it lofaruser/imaging-pipeline:latest
> NDPPP parsetfile
```

and frees the user (and sysadmins) from building all the required software packages. The following sections provide an overview of how to install and run LOFAR imaging pipelines using the docker.

Any questions concerning the docker image should be addressed to Science Data Center Operations (SDCO). Note however that **this docker framework is experimental and support for it from the SDCO group will be provided on a best effort basis**.

What is in the LOFAR docker image?

The docker image contains all the software packages that a user needs to run the LOFAR imaging pipelines like prefactor and factor. This includes the LOFAR offline software suite (containing tools like NDPPP and genericpipeline) along with other external tools like LoSoTo, AOFlagger, and WSClean. All packages included in this docker image are listed below:

- Casacore (version 2.4.1 including measures data)
- python-casacore (version 3.2.0)
- AOFlagger (version 2.14)
- Source finder pyBDSF (version 1.9.2)

- DPPP (version 4.1)
- LOFAR software (version 3.2.1)
- WSClean (version 2.8 including support for LOFAR primary beam and IDG)
- LoSoTo (release 2.0)
- RMextract (version 0.4)
- LSMTool (version 1.4.2)
- Dysco (version 1.2)
- Factor
- Python packages (including numpy, scipy, matplotlib)

Installing and setting-up docker

Docker is supported on various operating systems. Detailed instructions on how to install docker for different operating systems and cloud computing services can be found online.

In addition to installing the docker client, your system administrator should also add your username to the user group "docker". This can be achieved using the command "**sudo usermod -a -G docker <your username>**". Note that all users who wish to make use of the docker image on your machine must be added to the "docker" user group.

Once your sysadmin has setup the docker client on your machine and has added you to the docker group, you can download the LOFAR docker image using the command

```
docker pull lofaruser/imaging-pipeline:latest
```

Example: how to install and run docker on Fedora 27

On a fresh installation of Fedora, you need the following steps to install and run the docker image. Note that you need to have sudo privilege to carry out the steps listed below:

- Setup the repository using the commands "**sudo dnf install dnf-plugins-core**" and "**sudo dnf config-manager –add-repo https://download.docker.com/linux/fedora/docker-ce.repo**".
- Install the docker client with "**sudo dnf install docker-ce**".
- Start the docker service with "**sudo systemctl start docker**" and "**sudo systemctl enable docker**".
- Add your user to the docker group: "**sudo usermod -a -G docker <your username>**". Logout and login to apply the changes to your user.
- Run the command "**docker run –rm hello-world**" to verify your docker installation. If you see something like the following in your terminal, you have successfully installed docker on your machine.

```
$ docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
ca4f61b1923c: Pull complete
Digest: sha256:97ce6fa4b6cdc0790cda65fe7290b74cfebd9fa0c9b8c38e979330d547d22ce1
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://cloud.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/engine/userguide/
```

- Now, say, for example, you (as the sysadmin) want to provide access to a new user (say lof01). You would do "**sudo adduser lof01**" to create the user. Once the user has been created, you need to add lof01 to the usergroup docker using the command "**sudo usermod -a -G docker lof01**".

Once this is done, the user lof01 should be able to run the docker images.

What happens when I run the docker image?

The LOFAR docker image can be run using the following command

```
docker run --rm -it -v /home/temp/Data:/opt/Data lofaruser/imaging-pipeline:latest
```

As mentioned in the previous section, you can replace **lofar:latest** with **lofar:<version>** to run an older image. When the above command is run in a terminal, docker

- creates an interactive session and attaches it to the terminal (as indicated by the command line flag '-it' in the above command),
- initializes the LOFAR software environment, and

- maps the directory /home/temp/Data on your host machine to the directory /opt/Data/ inside the container.

Now, you (as **root** inside the container) can execute commands like NDPPP or wsclean inside the mapped directory (in this case /opt/Data/). If you do not want to be **root** inside the container, you can map the user on the host to the container as

```
docker run -u `id -u`:`id -g` -v /etc/passwd:/etc/passwd:ro -v /etc/group:/etc/group:ro -e
USER -v $HOME:$HOME -e HOME -w $HOME -e DISPLAY --net=host --rm lofaruser/imaging-
pipeline:latest
```

If you run the command **id**, you should see that the UID and GID of your user inside the container should be the same as that on the host.

How to run prefactor 3.0 inside the docker container?

In this section, we will demonstrate how to run the prefactor pipelines inside the docker container. For this example, we will follow the steps needed to run the calibrator part of prefactor, but the steps should be similar for the other pipelines that are part of prefactor. In this case, we will assume that the pipeline will be run from the directory /home/sarrvesh/dockertest/ and that all required measurement sets are available in /home/sarrvesh/dockertest/calibrator/. We will also assume that you have downloaded prefactor 3.0 from GitHub to the location /home/sarrvesh/prefactor/.

Now, start the docker container using the command

```
$ docker run -it -e USER -v $HOME/dockertest:$HOME/dockertest -e HOME -w $HOME --rm
lofaruser/imaging-pipeline:latest
```

Once inside the container, you need to source

```
$ source /opt/lofarsoft/lofarinit.sh
```

Navigate to the working directory /home/sarrvesh/dockertest

```
$ cd /home/sarrvesh/dockertest
$ ls -l calibrator
total 160
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB000_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB001_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB002_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB003_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB004_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB005_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB006_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB007_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB008_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB009_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB010_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB011_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB012_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB013_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB014_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB015_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB016_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB017_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB018_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB019_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB020_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB021_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB022_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB023_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB024_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB025_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB026_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB027_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB028_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB029_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB030_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB031_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB032_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB033_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB034_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB035_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB036_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB037_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB038_uv.MS
drwxr--r-- 17 9322 9000 4096 Apr  9  2018 L640803_SAP000_SB039_uv.MS
```

Next, we will create the working and runtime directories needed to run prefactor. Also, copy the **pipeline.cfg** file to the current directory.

```
$ mkdir working runtime
$ cp /opt/lofarsoft/share/pipeline/pipeline.cfg .
```

Edit the pipeline.cfg file so that the keys runtime_directory, working_directory, and log_file point to the correct locations inside the container.

```
runtime_directory = /home/sarrvesh/dockertest/runtime/
working_directory = /home/sarrvesh/dockertest/working/
log_file = /home/sarrvesh/dockertest/log/pipeline-%(job_name)s-%(start_time)s.log
xml_stat_file = /home/sarrvesh/dockertest/log/pipeline-%(job_name)s-%(start_time)s-
statistics.xml
```

You should also add the following lines to the end of the pipeline.cfg file

```
[remote]
method = local
```

Next, copy the Pre-Facet-Calibrator.parset

```
$ cp /home/sarrvesh/prefactor/Pre-Facet-Calibrator.parset .
```

and edit the following keys in the parset

```
! cal_input_path           =  /home/sarrvesh/dockertest/calibrator/
! cal_input_pattern        =  *.MS
! prefactor_directory      =  /home/sarrvesh/prefactor/
! losoto_directory         =  /opt/lofarsoft/
! aoflagger                =  /opt/lofarsoft/bin/aoflagger
```

For documentation on the other keys in parset, see prefactor documentation.

Finally, run the Pre-Facet-Calibrator.parset using genericpipeline as

```
$ genericpipeline.py -c pipeline.cfg Pre-Facet-Calibrator.parset
```

How to import the docker image inside singularity?

The docker image discussed in this chapter can be imported and converted into a singularity image using the command

```
singularity build lofar-pipeline.simg docker://lofaruser/imaging-pipeline:latest
```

This will create a new file called lofar-pipeline.simg. You can execute the container with

```
singularity run ./lofar-pipeline.simg
```

Once inside the container, you should source

```
source /opt/lofarsoft/lofarinit.sh
```

Alternately, you can also download the singularity image (containing the same packages as the docker image) from this link. Contact the SDCO if you have trouble running the singularity image.

FAQ

Where are the default RFI strategies stored in the docker container?

> The default RFI strategies (LBAdefault and HBAdefault) are stored in the directory /opt/lofarsoft/share/rfistrategies

Where can I find the pipeline.cfg file inside the docker container?

> The pipeline.cfg file that is needed for run prefactor is stored in the directory /opt/lofarsoft/share/pipeline

I get an "Illegal instruction error" when I run the docker image. What does this mean?

> This probably means that you are running an older/incompatible CPU. A docker image might have to be built on your machine. Please contact SDCO for further assistance if you encounter this.

Footnotes

[1] : This chapter is maintained by M. Iacobelli.

[2] : Docker is an opensource platform that makes use of container technology to create, deploy, and run applications easily. Detailed information about docker can be found here and elsewhere on the internet.